

# Model Evaluation, Explainability, and Fairness Reflection

## Hotel Booking Cancellation Prediction

Prepared by: Nusrat Jahan

Student ID: 101050785

Professor: Zahra Atf

Course: MBAI5310G AI Programming

---

### 1. Understanding the Business Problem

#### The Main Business Problem

Many hotels lose money when guests cancel their bookings. When a booking is cancelled close to the arrival date, the room often stays empty, because there is not enough time to sell it to someone else. This is direct lost income for that night, and it also makes planning harder, because a room that looked booked suddenly becomes free again. The hotel in this project wants to know, ahead of time, which bookings are likely to be cancelled, so it does not get caught by surprise. If the hotel can spot the risky bookings early, it can act in time, for example by confirming the stay with the guest, asking for a deposit, or carefully overbooking the busy nights. The aim of this work is to build a simple model that predicts whether a booking will be cancelled or kept, and then to test how accurate, clear, and fair that model really is, because a model is only useful if the hotel can trust it.

#### Why Predicting Cancellations Is Useful

Not every booking carries the same risk, and that is the reason a model is worth building. Some guests book early, change their plans, and cancel, while others book and almost always show up. If the hotel treats every booking the same way, it spends the same effort on a safe booking as on a risky one, which wastes time and money. A prediction model lets the hotel sort the bookings by risk, so the team can put its attention on the few bookings that are most likely to fall through. This is far more efficient than the usual approach of reacting only after a cancellation has already happened, when it is often too late to fill the room.

#### Decisions That Can Improve

A good cancellation model can support several real decisions, and each one saves money in a different way. For bookings that look risky, the hotel can send a friendly reminder, confirm the stay, or ask for a deposit, which lowers the chance of a quiet last minute cancellation. For bookings that look safe, the hotel can leave the guest alone and simply prepare for arrival, which avoids annoying loyal guests. The model can also guide the revenue team on how much to overbook on busy nights, since a steady share of risky bookings is likely to free up rooms. In short, the model turns a guessing game into a plan based on evidence.

### 2. Data Preparation

#### Dataset Overview

The dataset contains 350 booking records with 16 columns. Each row is one hotel booking and each column is one detail about that booking, such as the guest age, the lead time, the number of nights, the room price,

and whether the booking was cancelled. I checked for duplicate rows first, because repeated rows would make some bookings count twice and bias the model, and I found none. I then checked for missing values, because the model cannot work with empty cells. Only one column, the average daily rate, had missing values, 6 in total, and I filled them with the median price of about 257.24. I used the median rather than the average on purpose, because the median is a middle value that is not pulled up or down by a few very high or very low prices. The target column is whether the booking was cancelled. About 193 bookings were kept, close to 55 percent, and about 157 were cancelled, close to 45 percent. This near even split matters, because when the two answers appear about as often as each other, the model has a fair chance to learn both and is not tempted to simply always guess the more common answer.

## Feature Selection and Preprocessing

The target I wanted to predict was the cancellation result, so every other column was a possible input. I kept the booking details as inputs, such as guest age, booking channel, room type, number of guests, number of nights, lead time, average daily rate, special requests, previous cancellations, deposit type, and whether the stay covered a weekend, because these describe the booking itself. I removed the booking id column, because it is a unique label for each booking and holds no pattern the model could learn from. I also set aside three group columns, the age group, the region, and the guest type, and did not give them to the model. I did this on purpose, because I did not want the model to decide whether a booking will cancel based on the group a guest belongs to, which could be unfair. I kept these three columns only for the fairness check in a later section. Because a model can only work with numbers, I turned the text columns into number columns using one hot encoding, which creates a simple zero or one column for each category. After this step the input set had 15 columns. Finally I split the data, keeping 80 percent, or 280 bookings, for training and 20 percent, or 70 bookings, for testing, and I used a stratified split so that both parts keep the same balance of cancelled and kept bookings as the full dataset.

## 3. Building the Classification Model

I chose a Decision Tree model for this task, and the choice was deliberate. A Decision Tree is simple, it works well for a yes or no question like cancel or keep, and most importantly it is easy to explain to people who are not data experts, which matters for a business report. The tree learns by asking simple questions about a booking, such as whether the lead time is long or whether the guest has cancelled before, and it follows the answers down a series of branches until it reaches a final decision of cancel or keep. I limited the tree to a depth of four, which means it can ask at most four questions in a row before deciding. I chose a small depth on purpose, because a shallow tree stays simple and is less likely to just memorise the training bookings, a problem I check for later. I also set a fixed random state so the exact same tree is built every time the code runs, which makes the results repeatable. The model learns only from the 280 training bookings, and it is then tested on the 70 bookings it has never seen, which is the honest way to judge how it will behave on new bookings.

## 4. Evaluating the Model

### Evaluation Metrics

I measured the model with four scores, and each one answers a different question. The accuracy was about 0.71, which means the model predicted about 71 of every 100 test bookings correctly. Accuracy on its own can hide problems, so I also looked at precision and recall, which split the mistakes in two useful ways. The precision was about 0.72, which answers the question, when the model flags a booking as a cancellation, how often is it right, and the answer is about 72 times out of 100. The recall was about 0.58, which answers a different question, out of all the bookings that really cancelled, how many did the model catch, and here

it caught only about 58 of every 100. The gap between these two is the key point. The model is fairly trustworthy when it does raise a flag, but it lets a large share of real cancellations slip by without any flag. The F1 score, which is a single number that balances precision and recall, was about 0.64, sitting between the two as expected. For this hotel problem I treat recall as the most important score, and the reason is the business cost. The main goal is to catch the bookings that will cancel so the room can be sold again in time, and a missed cancellation is the expensive outcome. A recall of 0.58 means the model is missing close to half of the real cancellations, which is the clearest sign that it still needs work. The table below gathers the main results in one place.

| Result                                | Value |
|---------------------------------------|-------|
| Training Accuracy                     | 0.79  |
| Testing Accuracy                      | 0.71  |
| Precision                             | 0.72  |
| Recall                                | 0.58  |
| F1 Score                              | 0.64  |
| Correct kept bookings (True Negative) | 32    |
| False alarms (False Positive)         | 7     |
| Missed cancellations (False Negative) | 13    |
| Caught cancellations (True Positive)  | 18    |

Table 1: Summary of the main model results on the test data

## Confusion Matrix

The confusion matrix splits the 70 test results into four groups, which shows not just how many mistakes the model made but what kind. The model correctly kept 32 bookings and correctly flagged 18 cancellations, so it was right on 50 of the 70 bookings. It made 20 mistakes, and they are not the same kind. In 7 cases it flagged a booking as a cancellation when the guest actually kept it, which is a false alarm. In 13 cases it said a booking would be kept when the guest actually cancelled, which is a missed cancellation. The two correct groups are the largest, which is a good sign, but the fact that the missed cancellations (13) outnumber the false alarms (7) is the detail that matters most for the hotel. Figure 1 shows this as a coloured grid.

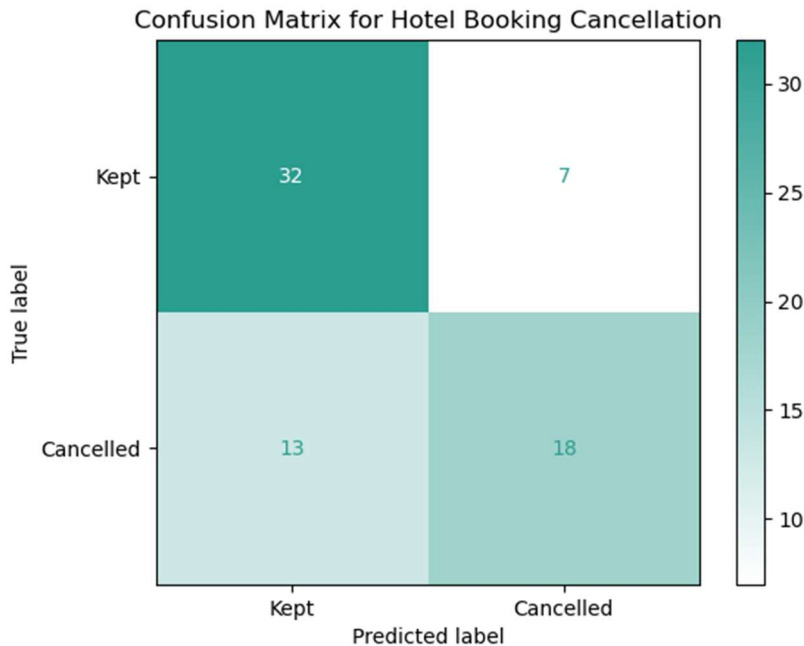


Figure 1: Confusion matrix for the hotel booking cancellation model

## Error Analysis

When I looked only at the mistakes, the most common one was the missed cancellation, where the model said a booking would be kept but the guest actually cancelled. There were 13 of these against only 7 false alarms, so missed cancellations make up almost two thirds of all the errors. This is the more costly kind of mistake for a hotel, and the reasoning is simple. When the model misses a cancellation, the hotel still expects that room to be filled, so it does not try to sell it again, and the room can sit empty at the last moment with no time to recover the income. A false alarm only wastes a little effort, but a missed cancellation loses a night of revenue, so reducing these 13 cases should be the first goal if the model is improved.

## 5. Checking Stability and Fit

### Cross-Validation

To make sure the result was not just luck from one lucky split, I tested the model five times on different slices of the data using cross validation, which trains and tests the model five times and reports five accuracy scores. The five scores were about 0.76, 0.77, 0.71, 0.67, and 0.64, giving an average close to 0.71 and a spread, measured as the standard deviation, of only about 0.05. The reason this matters is stability. The lowest fold at 0.64 and the highest fold at 0.77 are not far apart, and the average lands right on the single test score I found earlier, which tells me the 0.71 result is believable and not a one off. There is some movement between folds, which is normal for a small dataset of only 350 rows, but nothing that suggests the model swings wildly from one split to another.

### Overfitting and Underfitting

To check the fit, I compared how the model did on the data it learned from against the data it had never seen. The training accuracy was about 0.79 and the testing accuracy was about 0.71, a gap of roughly 0.07. A small gap like this points to mild overfitting, which means the model learned the training bookings a little better than it can repeat on new ones. To see the pattern clearly I tried tree depths from one to ten. As the depth grew, the training accuracy climbed steadily, from about 0.72 at depth one all the way to about

0.99 at depth ten, because a deeper tree can keep memorising more of the training data. The testing accuracy did the opposite. It was at its best at depth two, around 0.73, and then slowly drifted down into the high 0.60s at the deepest trees. This widening gap between the two lines is the textbook sign of overfitting, and it is shown in Figure 2. The model is not underfitting, because even a very simple tree still reached the low 0.70s rather than failing badly. The useful lesson is that a shallow tree of depth two would actually generalise a little better than the depth four tree I used, so here simpler is better.

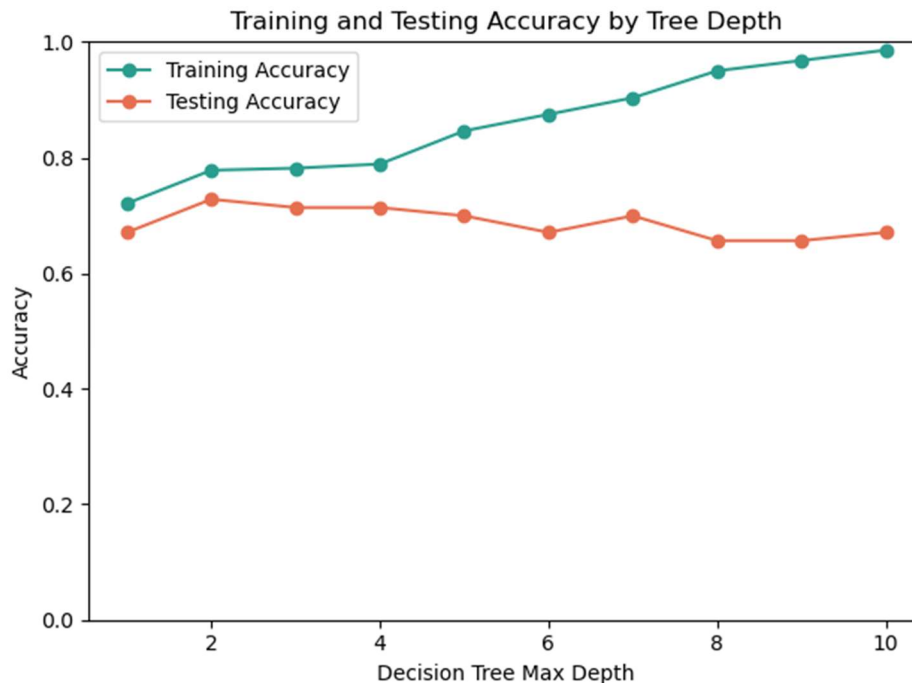


Figure 2: Training and testing accuracy across different tree depths

## 6. Explaining the Model

### Feature Importance

The model leaned on a small number of features, and the scores show just how unevenly. The most important by far was the lead time, the number of days between booking and arrival, with an importance score of about 0.53, which is more than half of all the importance on its own. Next came the number of previous cancellations at about 0.21, then the deposit type at about 0.09 and the room price at about 0.08, with everything else much smaller. So the model is really driven by two signals above all others, a long lead time and a history of cancelling. Several features, such as the booking channel and the room type, scored zero, which does not mean they are useless in general, only that this shallow tree did not need them once it had the stronger signals. One caution is important here. Importance shows what the model used to separate cancelled from kept bookings, not what causes a cancellation. A long lead time being useful to the model does not prove that booking early makes a guest cancel. Figure 3 shows the full ranking.

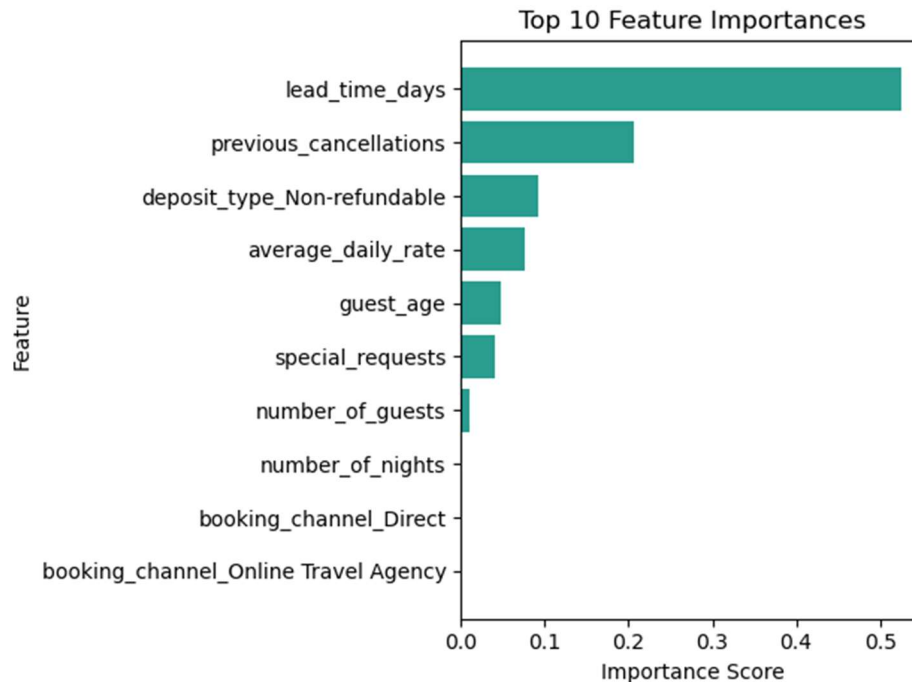


Figure 3: The most important features used by the model

### SHAP Explanation

SHAP is a tool that explains the model in more depth, both overall and for a single booking. Overall it agreed with the importance ranking that lead time has the largest effect, with previous cancellations, the room price, and the deposit type next, which gives me confidence that these are the real drivers. One small difference is worth noting. SHAP placed the room price just ahead of the deposit type, while the plain importance score had them the other way around. This kind of difference is normal, because the two methods measure influence in slightly different ways, and it is a good reason to look at more than one explanation rather than trust a single ranking. SHAP can also explain one booking. I took the first booking in the test set, which really was cancelled. The model starts from a base value of about 0.45, the average cancel chance, and for this booking it ended at about 0.60, so it predicted cancel. SHAP showed why, the long lead time of 181 days pushed the prediction up by about 0.28, which was by far the strongest push, while a few smaller features nudged it the other way but not enough to change the answer. The value of SHAP is that it shows direction, not just strength, so I can see which features pushed this booking toward cancel and which pushed it away.

### LIME Explanation

LIME explains one prediction in a simple, local way, by building an easy view around that single booking. For the same first booking, LIME also predicted cancel with a probability of about 0.60, which agrees with SHAP. It then listed the reasons with a size for each. The two strongest pushes toward cancel were that the booking had no non refundable deposit, contributing about plus 0.24, and that the lead time fell in the longer range of about 134 to 189 days, contributing about plus 0.11. The reasoning behind these makes sense, when there is no money lost by cancelling and the trip is still far away, a cancellation is easier. The main push the other way was that the guest had no previous cancellations, contributing about minus 0.15, which lowers the predicted risk. LIME is useful for the hotel team because it turns one prediction into a short list of plain reasons with clear sizes, which makes the decision easy to understand and to act on.

## 7. Fairness and Bias Reflection

Even though I did not use the group columns as model inputs, I still checked whether the model behaves the same for different groups of guests, because a model can pick up uneven patterns through the other features. The simplest way to spot a problem is to compare two numbers for each group, the real cancel rate, which is how often that group actually cancels, and the predicted cancel rate, which is how often the model says that group will cancel. When the predicted rate sits well below the real rate, the model is missing cancellations in that group. I also looked at the accuracy in each group, which is how often the model is right overall. I checked three groups, the age group, the region, and the guest type.

### By Age Group

By age group the accuracy ranged widely, from about 0.50 for the youngest guests aged 18 to 25 up to about 0.80 for guests aged 26 to 35. I am careful with the youngest group, though, because it had only 6 bookings in the test set, which is far too few to trust. With so few bookings, a single wrong prediction swings the score a lot, so its low accuracy may be chance rather than a real bias.

### By Region

By region the accuracy was fairly close, between about 0.68 and 0.74, but the gap between the real and predicted cancel rates told a clearer story. In the North the real cancel rate was about 0.47 while the model predicted only about 0.26, a wide gap that means the model is missing a good share of cancellations in the North. The other regions had much smaller gaps between their real and predicted rates, so this missing cancellation problem is mostly about the North rather than the regions in general.

### By Guest Type

| Guest Type | Bookings | Real Cancel Rate | Predicted Cancel Rate | Accuracy |
|------------|----------|------------------|-----------------------|----------|
| Business   | 13       | 0.46             | 0.46                  | 0.54     |
| Couple     | 17       | 0.53             | 0.53                  | 0.76     |
| Family     | 27       | 0.33             | 0.22                  | 0.74     |
| Solo       | 13       | 0.54             | 0.31                  | 0.77     |

*Table 2: Model behaviour by guest type*

The guest type results in the table above are worth reading carefully, because two different problems show up, and it helps to keep them apart. Solo guests are the clearest case of missed cancellations. Their real cancel rate is about 0.54, but the model predicted only about 0.31, a gap of more than twenty points, so the model is failing to flag many Solo cancellations even though its overall accuracy for them looks fine. Business guests show a different problem. Their predicted cancel rate, about 0.46, is the same as their real cancel rate, so on the whole the model predicts the right number of Business cancellations, yet their accuracy is the lowest of any group at about 0.54. This means the model is getting the right total but the wrong individual bookings, mixing up which Business guests cancel. Couple and Family guests are handled best, with accuracy around 0.74 to 0.77 and smaller gaps. So when I say the model misses cancellations for a group, the strongest example is Solo, while Business is better described as low overall accuracy, and these are two different fairness concerns. Figure 4 shows the same picture.

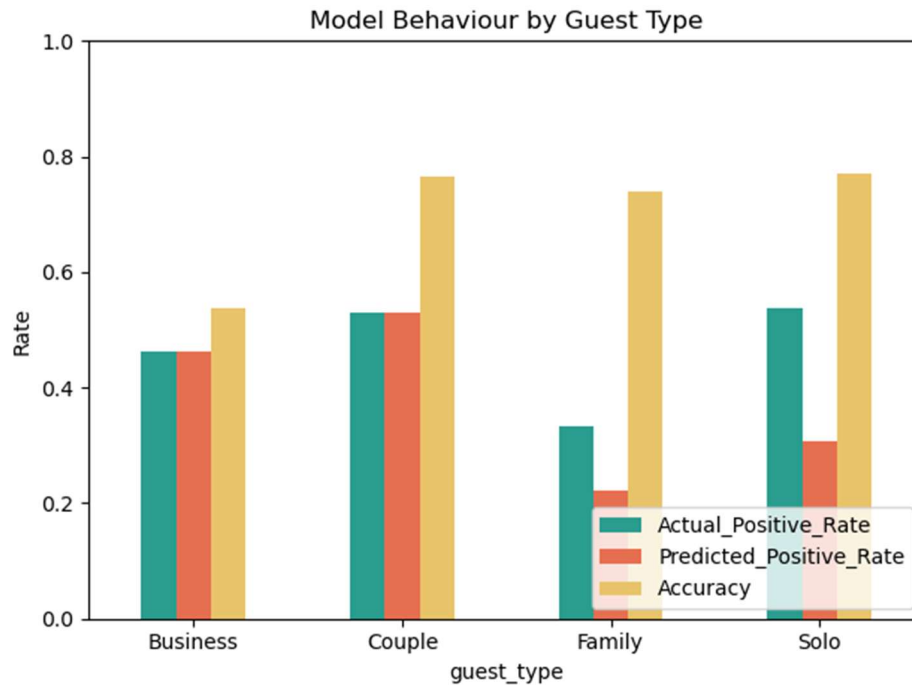


Figure 4: Real cancel rate, predicted cancel rate, and accuracy by guest type

These gaps matter, because if the hotel acted on the model alone, some groups would have their cancellations missed more often than others. Solo guests and guests in the North would slip through more, while Business bookings would simply be judged less reliably. Some of the gaps come from very small group sizes, so the exact numbers should be read with care, but the direction of the differences is real and worth watching. This is why the model needs a fairness review before it is trusted for real decisions, even though the group columns were never used as inputs.

## 8. Business Recommendation and Final Interpretation

### Patterns Discovered

The clearest pattern is that bookings made a long time in advance, and guests who have cancelled before, are the most likely to cancel again, since the lead time and the previous cancellations together hold most of the model's importance. The deposit type and the room price add smaller signals. On performance, the model is moderately accurate at about 0.71 and reasonably stable across the cross validation folds, but its recall of about 0.58 shows it tends to miss real cancellations more often than it raises false alarms.

### How the Business Can Use the Model

The hotel can use the model as an early warning tool rather than an automatic decision maker, and the numbers support that careful use. Bookings the model flags as risky, especially those with long lead times and a history of cancellations, can be passed to staff for a quick review, who may confirm the stay, ask for a deposit, or plan for the room to come free. For bookings the model judges safe, the hotel can simply prepare for arrival. Because the model is right about 72 percent of the time when it does flag a booking, its flags are worth a look, but because it misses many real cancellations, staff should not assume that an unflagged booking is guaranteed to stay.

### Final Recommendation



Taking everything together, this model is not ready to be used on its own for real decisions, and the evidence is consistent. Its testing accuracy is moderate at about 0.71, it shows mild overfitting with a training to testing gap of about 0.07, it misses close to half of the real cancellations, and it does not behave equally across guest groups. For these reasons it is best treated as an early prototype. It already gives useful insight into which bookings are risky and which features matter most, but before it supports real decisions it needs improvement, such as more data, better features, tuning of the tree depth, a proper fairness review, and a comparison with stronger models like Logistic Regression or Random Forest. Used as a support tool alongside human judgment, however, it can already help the hotel start managing cancellation risk in a smarter way.

## 9. Limitations and Responsible AI Reflection

### Dataset Limitation

The dataset is small, with only 350 bookings, and it appears to be synthetic, so it may not match the messy patterns of real hotel data. The small size is also why several fairness groups had only a handful of bookings, which makes their numbers shaky. I filled the missing room prices with the median, which is a safe choice but still an estimate that can add a little noise. In a real setting the patterns would likely be more complex and shaped by outside factors such as season, local events, weather, and price changes, none of which are in this dataset.

### Why the Model Is Not Perfect

A Decision Tree also has natural limits that explain some of the errors. It can overfit by memorising the training data, which is exactly what the depth test showed at higher depths. It makes its decisions with simple straight cut rules, so it can miss smoother or more complex relationships in the data. It is also sensitive to small changes in the data, which is one reason the accuracy moved a little across the five cross validation folds. These limits are part of why the model misses some real cancellations, and why a stronger model might do better.

### Risk of Bias Without Human Review

If the hotel used the model to apply strict rules automatically, for example refusing flexible booking to every guest the model flags, it could treat some guests unfairly. The fairness check showed the model misses cancellations more often for some groups, most clearly Solo guests and guests in the North, where the predicted cancel rate sat well below the real one. Acting on the model alone could mean these groups are served less well, or that some guests are wrongly labelled as risky because of patterns the model picked up from the other features rather than from their real intentions.

### Why Human Judgment Still Matters

The model is a tool that points to where the risk may be, but it cannot understand the full story behind any one booking. A staff member can see things the model cannot, such as a guest who always keeps their bookings, a special event that explains a long lead time, or a deposit that was already paid. The model should highlight the bookings worth a closer look, while the final decision stays with the people who know the guests and the business. Using the model results without human review also risks repeating old patterns in future decisions, so a person should always stay in the loop.